# Rushing at SPDZ: On the Practical Security of Malicious MPC Implementations

Alexander Kyster*, Frederik Huss Nielsen*, Sabine Oechsner† and Peter Scholl*
*Aarhus University, Denmark
peter.scholl@cs.au.dk
†Vrije Universiteit Amsterdam, The Netherlands
s.a.oechsner@vu.nl

***Abstract*—Secure multi-party computation (MPC) enables parties to compute a function over private inputs while maintaining confidentiality. Although MPC has advanced significantly and attracts a growing industry interest, open-source implementations are still at an early stage, with no production-ready code and a poor understanding of their actual security guarantees.**

**In this work, we study the real-world security of modern MPC implementations, focusing on the SPDZ protocol (Damgård et al., CRYPTO 2012, ESORICS 2013), which provides security against malicious adversaries when all-but-one of the participants may be corrupted. We identify a novel type of MAC key leakage in the MAC check protocol of SPDZ, which can be exploited in concurrent, multi-threaded settings, compromising output integrity and, in some cases, input privacy. In our analysis of three SPDZ implementations (MP-SPDZ, SCALE-MAMBA, and FRESCO), two are vulnerable to this attack, while we also uncover further issues and vulnerabilities with all implementations. We propose mitigation strategies and some recommendations for researchers, developers and users, which we hope can bring more awareness to these issues and avoid them reoccurring in future.**

## 1. Introduction

Secure multiparty computation (MPC) allows mutually distrusting parties to compute a public function of their private inputs while hiding their inputs from each other.

In recent years, MPC has gained interest due to its ability to add privacy to virtually any computation, including privacy-preserving machine learning [1], secure auctions [2], collaborative statistics, protecting cryptographic keys and more [3].

Academic prototype implementations have come a long way to demonstrate the feasibility of executing MPC protocols in many different contexts, from humble beginnings like the Fairplay compiler [4] to highly optimized and practically efficient, modern MPC compilers such as MP-SPDZ [5]. There is now a growing interest in MPC from industry, from both large and small companies, as evidenced by the range of partners in the MPC Alliance[1]. While the only publicly available MPC implementations are currently prototypes that do not claim to be production-ready, we expect them to form the basis of future production implementations due to the inherent complexity of MPC protocols.

As a result, MPC is on the verge of transitioning from academic prototypes to production-ready implementations. To achieve the desired efficiency, implementations heavily rely on optimization techniques such as multi-threading.

Modern MPC protocol designs are usually accompanied by a formal security analysis. However, as is typical with complex, state-of-the-art cryptographic protocols, implementations are difficult to get right and may not always strictly follow the original specification. This could be for various reasons, including ambiguities in protocol descriptions in research papers, programming errors, or a perceived need for new optimizations not covered by the original security analysis. The situation is complicated further by the complexity of MPC protocols, which involve many rounds of communication and often use a large number of cryptographic building blocks such as oblivious transfer, commitment, zero-knowledge proofs, and homomorphic encryption. In this work, we therefore ask the question:

> *How secure are today's MPC implementations in practice, and what are the likely pitfalls?*

While there is some literature on side-channel and fault attacks on MPC protocols [6], [7], [8], to the best of our knowledge, there have been no prior, in-depth studies of the real-world security of general-purpose implementations in the standard attack models which the protocols were explicitly designed to withstand.

While MPC is a broad field and we cannot hope to provide satisfactory answers immediately, we initiate the study of this topic by focusing on protocols in the malicious security model with a dishonest majority, where up to $n-1$ out of $n$ parties can be actively corrupted by an adversary. This focus is motivated by the fact that maliciously secure protocols are highly desirable for their strong security guarantees, but at the same time, are typically more complex and challenging to implement securely. This is reflected, for instance, in the multitude of attacks on zero-knowledge proof implementations [9], [10].

1. https://www.mpcalliance.org/

## Our Contributions

We chose to focus our attention on implementations of SPDZ [11], [12], a popular family of maliciously secure MPC protocols in the dishonest majority setting. SPDZ is divided into two phases: a preprocessing phase, where the parties generate correlated randomness, and an online phase, where the actual computation takes place. We focus on the online phase, as it is a much simpler protocol that doesn't rely on too many building blocks. Furthermore, even without a preprocessing protocol, the online phase is still useful in a setting with a non-colluding dealer party, who is trusted to generate and distribute the correlated randomness.

**MAC Key Leakage in SPDZ.** Our main contribution is to identify a previously unknown type of leakage in the SPDZ protocol: An attacker can take advantage of the MAC check protocol to learn the global MAC key used to authenticate secret shared values, at the cost of aborting the protocol execution. Note that our findings do not contradict the existing SPDZ security analysis, and we discuss how the leakage is in fact implictly permitted by existing MPC security models. In a nutshell, the leakage cannot be exploited in a sequential execution, the standard model for the security analysis of cryptographic protocols.

**Attacking integrity and privacy via MAC key leakage.** While this leakage does not impact the security of SPDZ as described in the literature, we observe that it can be exploited in concurrent implementations that use multi-threading as an optimization[2]. In particular, an implementation where two different threads are simultaneously running an instance of the MAC check protocol may be vulnerable, and this would allow a malicious party running a modified version of the prescribed binary, to fully control the outputs of the computation in one thread, breaking output integrity. In some instances, this can also violate input privacy. Such a scenario could arise, for instance, in an application where parties perform queries to a secret database, or private order matching in a financial market. These use-cases are likely to benefit from multi-threading in order to parallelize the processing of queries or orders.

**Survey of SPDZ Implementation Security.** With our vulnerability in mind, we survey the main existing SPDZ implementations. We identify three implementations that are currently in use (MP-SPDZ [5], SCALE-MAMBA [13] and FRESCO [14]) and demonstrate that two out of three are vulnerable to this attack. Furthermore, we discovered additional vulnerabilities and programming errors in all three implementations. These include the absence of MAC checks, inappropriate use of hash-based commitments, and other weaknesses that can potentially lead to unintended consequences for users.

2. Similar concerns hold for cooperative multitasking as well, but we focus on multi-threading for the remainder of this work.

**Disclosure.** We notified the developers of the affected implementations of the vulnerabilities. The issues in MP-SPDZ and FRESCO have subsequently been patched; we present the attacks on the unpatched versions. The SCALE-MAMBA framework is no longer actively maintained, and has not been patched.

**Mitigation.** We discuss various mitigations for the SPDZ-specific vulnerabilities, including simple programming fixes, design choices and protocol modifications. We also conclude by making some recommendations for MPC researchers, developers and users.

**Follow-Up: Collecting MPC Implementation Pitfalls.** As a result of discussions around this work during the RWMPC 2025 workshop, a collection of known pitfalls in MPC has been initiated[3]. Beyond the pitfalls discussed in this work, the collection currently includes mistakes in instantiating common network assumptions and improper use of various cryptographic primitives.

## 1.1. Related Work

While we are not aware of any other work that analyses the security of real-world MPC implementations in the standard attacker model, there are a number of works that consider side-channel attacks not covered by the standard security analysis that accompanies modern MPC protocols. Levi and Hazay [6] demonstrated that the FreeXOR optimization [15] for garbled circuits [16] is almost trivially vulnerable to side-channel attacks. Hashemi et al. further reported timing side channel attacks on garbled circuit protocols [7] as well as fault attacks on secure two-party neural network inference based on garbled circuits [8], while Shen and Jin demonstrated a side-channel attack on the encryption scheme used in garbled circuit protocols [17].

At the same time, related types of cryptographic protocols that are already in production such as zero-knowledge proofs and threshold signing or key generation protocols have been the subject of some scrutiny. David et al. [18] reviewed a proposed MPC ceremony for distributed RSA key generation, and discovered a number of implementation issues that could have led to severe vulnerabilities in a deployment. Multiple threshold ECDSA implementations have been found vulnerable to selective abort attacks, which have some similarity to our MAC key leakage attack: van de Pol [19] reports a selective abort attack on a threshold ECDSA implementation based on oblivious transfer extension; Makriyannis, Yomtov and Galansky [20] report another selective abort attack on real-world deployments of Lindell's two-party threshold ECDSA protocol [21]. A difference to our attack is that in both of the above attacks, only 1 bit of information is leaked after each selective abort, so the attacker must be able to abort many times without being detected to gain a significant advantage. On the other hand,

3. https://github.com/rot256/mpc-pitfalls

our attack leaks the entire MAC key at once, and potentially before a single abort has even been detected.

Zero-knowledge proof systems that use weak instantiation of the Fiat-Shamir transform — as well as their applications — have been known to be vulnerable to adaptive soundness attacks for more than a decade, e.g. the voting protocols Helios [22] and sVote [9]. Surprisingly, many current zero-knowledge implementations still use weak Fiat-Shamir [10], which the authors speculate might be due misunderstandings about the specification.

### 1.2. Outline

Section 2 introduces the relevant background on MPC protocols and MPC implementations as well as the SPDZ protocol. In Section 3, we show how the SPDZ protocol can leak the global MAC key, and how this could be turned into an attack in a concurrent implementation. We survey SPDZ implementations with respect to this attack in Section 4, and then discuss mitigation strategies in Section 5, including a masked MAC check protcol which is analyzed further in Appendix B. Section 6 presents further vulnerabilites that we found in the surveyed SPDZ implementations. Finally, we conclude in Section 7 with general recommendations to researchers and implementers.

## 2. MPC Background

In this section, we review background on secure multiparty computation in the dishonest majority setting, the SPDZ protocol and the architecture of SPDZ implementations.

### 2.1. MPC Overview

In secure multiparty computation, $n$ mutually distrusting parties $P_1, \ldots, P_n$ with secret inputs $x_1, \ldots, x_n$, respectively, wish to jointly evaluate a public function. This function is typically represented as a circuit (Boolean or arithmetic). The computation may be reactive, i.e. after outputting a value, the computation can resume, possibly with additional inputs. Intuitively, an MPC is secure if no party can learn anything about the inputs of other parties, except for the information that the output leaks about them.

**Network Model.** The standard assumption in the MPC literature is that the parties communicate over pre-established secure communication channels, either broadcast or point-to-point. A secure channel provides both confidentiality and authentication of messages. Protocols are often described in a synchronous model, consisting of a number of sequential rounds of interaction, where in each round, every party sends and receives a message to and from every other party. In a synchronous protocol, there is implicitly a known upper bound on the network delay, so that if in any given round, a party does not receive an expected message from some other party, the receiving party can assume the sender is corrupted (and abort the protocol if needed).

**Corruption Model.** When we talk about corruption in the context of MPC, we specifically mean insider corruption where the adversary takes control of a party and their entire local state. The adversary can also observe all (encrypted and authenticated) communication on the network. Under semi-honest corruption, a corrupted party outwardly still follows the protocol specification but may run additional computation internally. In the case of malicious corruption, which we focus on, a corrupted party can behave arbitrarily. Note that, in a round-based protocol where parties are instructed to send messages simultaneously, a corrupted party can always be a *rushing adversary*, who waits to read messages from the honest parties, before sending its own message for the same round.

Security guarantees are typically specified with respect to a corruption threshold $t$, the maximum number of corrupted parties the protocol can tolerate while maintaining security. We will particularly focus on the dishonest majority case with $t = n - 1$ for an $n$-party protocol.

**Protocol Idea.** In a typical protocol, each party $P_i$ first encodes their input $x_i$ (e.g. through encryption or secret sharing). Then for each gate, the parties run a protocol to compute the encoded gate output based on the encoded gate inputs. For stronger attacker models, an additional mechanism might be needed to ensure that parties did not deviate from the protocol. For the sake of security analysis, the standard assumption is that the the MPC protocol evaluates the circuit gate-by-gate, in a sequential way.

### 2.2. Secret Sharing and MACs

In the dishonest majority setting, when up to $n - 1$ out of $n$ parties may be corrupted, most MPC protocols rely on additive secret sharing, where a secret $x \in \mathbb{F}$, over some finite field $\mathbb{F}$, is divided into shares $x_1, \ldots, x_n \in \mathbb{F}$, which are randomly sampled such that as $x = x_1 + \cdots + x_n$.

Since additive shares can be easily changed without detection, to protect against malicious parties, we can use a form of authenticated sharing to reliably open shares. This can be done using information-theoretic message authentication codes (MACs), and comes in two main variants, depending on whether the *shares* or the *values* are authenticated.

**Pairwise MACs.** With pairwise MACs, each party $P_i$ holds a global MAC key $\alpha_i \in \mathbb{F}$, and for each value $x$ that is authenticated, an additional set of keys $\{\beta_{i,j}^{(x)}\}_{j \neq i}$ that is independently sampled[4]. Party $i$ holds the values:

$$\left(\alpha_i, x_i, \{\beta_{i,j}^{(x)}, \tau_{i,j} := \alpha_j \cdot x_i + \beta_{j,i}^{(x)}\right)$$

Now, when opening the shares of $x$, each party must send its MAC tag $\tau_{i,j}$ to every other party $P_j$, along with its share $x_i$. $P_j$ then checks the tags of all the shares it

4. The superscript in $\beta_{i,j}^{(x)}$ refers to $x$ as an identifier or variable name, rather than the secret value.

receives using its keys $\alpha_j, \beta_{j,i}^{(x)}$. It can be shown that any party who tries to send an incorrect share will be caught with probability at least $1 - 1/|\mathbb{F}|$.

These MACs are set up with the help of a preprocessing phase that distributes correlated randomness among the parties. They have been used in the BDOZ protocol [23] for arithmetic circuits, as well as the TinyOT protocols [24], [25], [26], [27] for Boolean circuits.

**Secret-shared MACs.** Another approach is to authenticate $x$ instead of the shares $x_i$. Now, parties hold additive shares $\gamma_i$ of a tag on $x$ defined by $\gamma = \alpha \cdot x$, where $\alpha \in \mathbb{F}$ is a global MAC key additively shared among the parties. Concretely, we represent this by the following notation:

$$[\![x]\!] := \big((x_i, \gamma_i)\big)_{i=1}^{n}, \qquad \text{where } \sum_i x_i = x, \sum_i \gamma_i = \alpha x$$

where party $P_i$ holds $(x_i, \gamma_i)$ and additionally has a share $\alpha_i$ of the global MAC key $\alpha = \sum_i \alpha_i$.

This form of MAC was introduced in the SPDZ protocol [11], [28]. The procedure for checking the MAC on an opened value is a little more complex than pairwise MACs, as we will see shortly. The main advantage is that each party stores just two field elements per shared value instead of $n$.

## 2.3. SPDZ Protocol

SPDZ is an actively secure MPC protocol that tolerates up to $n-1$ out of $n$ corrupted parties, based on secret-shared MACs. Basic computations in SPDZ can be represented as arithmetic circuits over a finite field $\mathbb{F}$, which is typically either a large prime field $\mathbb{F}_p$ with $p \approx 2^\lambda$, or a binary extension field $\mathbb{F}_{2^\lambda}$, where $\lambda$ is a statistical security parameter. These computations can also be reactive, meaning that after outputting some value, the parties can continue to do further computation that may depend on this output, and later output additional values as desired.

The protocol consists of a preprocessing phase, which generates multiplication triples and other correlated randomness, and an online phase where the computation takes place.

**Historical Note: SPDZ and its Variants.** In this work, we focus on the version of the SPDZ online phase as described in [28]. This extends the original SPDZ protocol [11] — which only supports evaluation of a single circuit — by allowing for reactive computations. There are many other, subsequent variants of SPDZ, including MASCOT [29], Overdrive [30], TopGear [31] and SPDZ2k [32]. However, most of these variants use the same online phase as [28], and only differ in the preprocessing phase, which is the more expensive part of the protocol. Even SPDZ2k, which uses a different MAC representation, has a similar type of MAC check protocol as [28], and most of our findings are also applicable to SPDZ2k.

**Computing on Authenticated Secret Shares.** During the SPDZ online phase, each input and intermediate value of the computation will by held by the parties as an authenticated sharing $[\![x]\!]$.

Any linear operation over $\mathbb{F}$ can easily be performed to secret-shared values via local computation on the shares, thanks to linearity of the MAC scheme. Addition with a constant $c$ can be done by defining the sharing $[\![c]\!] := ((c, \alpha_1 \cdot c), (0, \alpha_2 \cdot c), \ldots, (0, \alpha_n \cdot c))$.

To multiply two secret-shared values, SPDZ uses a preprocessed, random *multiplication triple*, which consists of shares $([\![a]\!], [\![b]\!], [\![c]\!])$, where $a, b \in \mathbb{F}$ are uniformly random and $c = a \cdot b$. Given this, two values $[\![x]\!], [\![y]\!]$ can be multiplied by first locally computing shares of $d = x - a, e = y - b$, then opening $d$ and $e$ and computing an appropriate linear combination. The multiplication triples are generated using a preprocessing protocol, typically based on homomorphic encryption or oblivious transfer. It's crucial that each triple is only used once, to ensure that no information on $x$ and $y$ is revealed.

**Opening and Checking MACs.** Opening secret-shared values in SPDZ is needed both when using multiplication triples and also to output a (public) result of the computation. Since the computation may be reactive, we need to be able to repeatedly check MACs on results of the computation without revealing the shared MAC key $\alpha$.

To perform an opening, the parties first send their shares $x_i$ and reconstruct $x$. This step may be faulty, and is sometimes called *partial opening*. To verify that the opening was performed correctly, the parties can run a MAC check protocol for $x$, shown in Figure 1. Since the MAC check takes several rounds of communication and can be optimized when batched on many openings at once, it is advantageous to defer the MAC checks until just before any opening which depends on secret inputs. In a batch check, instead of committing to a $\sigma_i$ value for each opening being checked, the parties can first run a coin-tossing protocol to obtain a random challenge, and use this to take a random linear combination of the values being checked, which is then checked using Figure 1. We detail the batch check protocol in Appendix A.

It's important to note that, while the openings of the $d$ and $e$ values in the multiplication protocol can always be safely postponed, more care must be taken when outputting results of a computation. Firstly, before starting to open some output $[\![z]\!]$, to prevent any undesired information leakage, the parties must first verify all prior openings (typically, from multiplications) that were involved in the computation of $z$. Secondly, the result of the opening of $z$ should be checked immediately, before the parties accept its result and continue the computation or use it in any external protocol.

**UC Security Guarantees.** SPDZ was shown to be secure in the Universal Composability (UC) framework [33], the current gold standard for cryptographic protocol security. The analysis [11] shows essentially that the SPDZ online phase is a secure MPC protocol under static active corruption of up

$\Pi_{\mathsf{CheckMAC}}(x, [\![x]\!])$

1) Party $i$ parses its share of $[\![x]\!]$ as $(x_i, \gamma_i, \alpha_i)$
2) Each party broadcasts a commitment to $\sigma_i := x\alpha_i - \gamma_i$
3) After receiving all other parties' commitments, each party opens their commitment to reveal $\sigma_i$
4) If any opening fails, abort.
5) If $\sum_i \sigma_i = 0$, output success, otherwise abort.

Figure 1: SPDZ MAC check protocol

to $n-1$ parties, even when run concurrently with arbitrary other protocols. This assumes a secure implementation of the ideal functionality for the preprocessing phase, as well as a UC-secure commitment scheme for use in the MAC check protocol. We will discuss this security analysis and its limitations in more detail in Section 3.3.

### 2.4. MPC Implementations

An MPC implementation takes as input a representation of the computation to be executed in MPC as well as parameters such as the number of parties, and outputs code for each party. The input might first be compiled to an intermediate bytecode representation. The parties can then run the code to perform the secure computation on their private inputs. A specific MPC framework can support implementations of multiple MPC protocols and let the user specify which one to use to perform the computation.

**Specifying MPC Tasks.** When implementing an MPC protocol such as SPDZ, one of the major design decisions is how a user specifies the computation task that is to be run by the protocol. Perhaps the most basic approach is to use a simple (Boolean or arithmetic) circuit format that is provided by the user, and directly parsed and executed by the protocol. A more sophisticated version of this it to provide a domain-specific language that allows the user to specify the computation in a more high-level way.

Implementations that use a high-level language typically come with some form of MPC compiler that translates the function description into an intermediate representation. This is aimed to allow for easier specification of the function to compute by the user, as well as program optimizations such as limited automated parallelisation, before executing the code in the underlying MPC protocol.

A final, more direct approach, is to provide library functions that implement the basic gates supported by the protocol, and have the user implement the desired functionality by calling these functions.

## 3. MAC Check Leakage Attack

In general, multi-threading is a powerful tool for any type of computation allowing the programmer to take advantage of the multiple cores of the CPU to increase efficiency, but doing so requires extra care to avoid efficiency and security issues when said threads access the same information. Our first attack allows a malicious party to learn the secret MAC key $\alpha$ in one thread of a program and then use $\alpha$ to forge openings in a second thread. We first describe how to leak MAC keys even in single-threaded SPDZ implementations (where this leakage does not seem to affect security) before explaining how to exploit this leakage in a setting with multi-threading. In Section 4, we discuss and demonstrate the applicability of our attack in various implementations.

### 3.1. SPDZ MAC Failure Leaks the MAC Key

Consider a protocol execution between $n$ parties, and a point in the protocol where the parties want to open some secret-shared value $[\![x]\!]$. This opening involves the MAC check protocol in Figure 1. We first observe that a malicious party can learn the secret MAC key during the MAC check protocol, at the cost of not completing the protocol.

**Honest MAC Check.** Recall that at this point, the $i$-th party holds a share $x_i \in \mathbb{F}$ and MAC share $\gamma_i \in \mathbb{F}$, with $\sum_{i=1}^n x_i = x$ and $\sum_{i=1}^n \gamma_i = \alpha x$. After opening $x$ by sending their shares $x_i$, the parties check the MAC on $x$ without revealing $\alpha$, using the following commit-and-open protocol [28]: Party $i$ first commits to the value $\sigma_i = x\alpha_i - \gamma_i$, then all commitments are opened, and the parties check that $\sum_i \sigma_i = 0$.

**Leaking the MAC Key.** The standard analysis of the MAC check [28] shows that if any set of corrupted parties tries to open $x$ incorrectly, then they need to guess $\alpha$ in order to pass the check. However, it's also easy to see that if $x$ is opened incorrectly then at the end of the protocol *the corrupted parties can also compute* $\alpha$! This can be seen as follows:

- When opening $x$, a corrupted party $P_i^*$ sends share $x_i' = x_i + 1$. The parties reconstruct $x' = x + 1$.
- In the MAC check, each honest $P_j$ sends a commitment to $\sigma_j = x'\alpha_j - \gamma_j$.
- The corrupt $P_i^*$ computes $\sigma_i = x'\alpha_i - \gamma_i$, and sends a commitment to an arbitrary value.
- On receiving the honest parties' openings, compute

$$\sum_i \sigma_i = x'\alpha - x\alpha = \alpha$$

Thus, a corrupt party learns the MAC key by just adding 1 to its share of $x$. Of course, this point can only be reached *after* committing to the $\sigma_i$ values, so there is no way to use this leakage to cheat in the MAC check itself in this concrete MAC check instance. Indeed, if $P_i^*$ opens its commitment to $\sigma_i$, it will be detected except with probabilty $1/|\mathbb{F}|$. However, if the adversary never opens their commitments, then the honest parties will be left waiting and may not (immediately) detect that something has gone wrong.

### 3.2. Attack Description

To try to exploit the MAC key leakage, we consider a setting where several, independent MAC checks are performed in parallel in different threads of computation. Then, the MAC key $\alpha$ can be extracted in one thread, and the adversary can subsequently stall that thread by not sending its commitment openings, leaving the honest parties hanging. At this point, it can use $\alpha$ to manipulate the execution in the other thread.

**Example Computation.** To illustrate the dangers of this kind of attack, we will consider throughout, a simple example of a program that computes a product of secret-shared values. In a general, $n$-party protocol, we focus on the behaviour of an honest party $P_1$ and a malicious party $P_2^*$, who provide inputs $y$ and $x$, respectively. Further, suppose that during the computation, the parties at some point wish to compute and reveal the result of the randomized product:

$$f_{mult}(x, y) := (\llbracket x \rrbracket \cdot \llbracket r \rrbracket) \cdot \llbracket y \rrbracket. \tag{1}$$

Here, $\llbracket r \rrbracket$ is a secret-shared, authenticated random mask, unknown to all parties, which is uniformly sampled from the field $\mathbb{F}$ in the preprocessing phase. Computing the above could, for instance, be useful if the parties want to check that at least one of the inputs $x$ and $y$ is zero, as part of a larger computation. Since $\mathbb{F}$ is large, $r$ is non-zero with overwhelming probability so nothing more is revealed about the inputs $x$ and $y$.

Suppose that for each party, the protocol for computing $f_{mult}$ is executed in a local thread $T_2$, while some other function is computed in a different thread $T_1$. If $T_1$ contains a MAC check, then $P_2^*$ can learn the MAC key $\alpha$ in $T_1$ and then use this to cheat in $T_2$, as follows: If $P_2^*$ sets their input as $x = 0$, the product $(x \cdot r)$ evaluates to 0. Now, $P_2^*$ can adjust their share of $\llbracket x \cdot r \rrbracket$ by $+1$ and add $\alpha$ to the corresponding MAC. As a result, the MAC check in $T_2$ will pass, and the result in this thread is $(0 \cdot r + 1) \cdot y = y$, party $P_1$'s secret input.

This example illustrates the well-known fact that in maliciously secure MPC protocols, the properties of *correctness* and *privacy* cannot easily be decoupled. Indeed, while SPDZ MACs seem to serve to ensure correctness, violating correctness can easily lead to a breach of privacy.

### 3.3. Limitations of Existing Security Analysis

After seeing the attack above, one may wonder how to interpret it. After all, SPDZ was shown to be secure in the Universal Composability framework, the gold standard for cryptographic protocol security. As we will see, our findings do not invalidate the existing security analysis, but rather point to an interesting gap between current models and implementations in practice.

**3.3.1. The UC Framework.** Universal composability [33] is a framework for proving the security of cryptographic protocols. The intended security guarantee of a protocol is described through an *ideal functionality*, a trusted party that performs the protocol task on behalf of the parties. The ideal functionality can be thought of a specification of the intended behaviour of the protocol that is oblivious to any cryptographic implementation details.

An important feature of the UC framework is its security guarantee under "concurrent execution": A UC-secure protocol instance remains secure even when executed in the presence of arbitrary independent protocols running concurrently.

**3.3.2. SPDZ Security in UC.** SPDZ was shown secure with respect to the "arithmetic black box" functionality $\mathcal{F}_{\mathsf{ABB}}$, shown in Figure 2. In a nutshell, the arithmetic black box allows parties to store input values in memory and perform arithmetic field operations over $\mathbb{F}$ (addition, multiplication) on them. It captures the (reactive) evaluation of arithmetic circuits in a way that is oblivious to implementation details such as the use of secret sharing or MACs.

**Functionality $\mathcal{F}_{\mathsf{ABB}}$**

**Input:** On receiving $(\mathsf{input}, id, P_j, x)$ from party $P_j$ and $(\mathsf{input}, P_j)$ from all other parties, where $x \in \mathbb{F}$, store $(x, id)$ in memory.

**Add:** On receiving $(\mathsf{add}, id_1, id_2, id_3)$ from all parties:

1) Retrieve $(id_1, x)$ and $(id_2, y)$
2) Store $(id_3, x + y)$

**Multiply:** On receiving $(\mathsf{mult}, id_1, id_2, id_3)$ from all parties:

1) Retrieve $(id_1, x)$ and $(id_2, y)$
2) Store $(id_3, x \cdot y)$

**Output:** On receiving $(\mathsf{output}, id)$ from all parties, where $(y, id)$ has previously been stored in memory:

1) Leak $y$ to the adversary.
2) Wait for a message from the adversary; if it sends $\mathsf{abort}$ then abort and output $\perp$ to all parties. Otherwise, if it sends $\mathsf{deliver}$ then send $y$ to all parties.

Figure 2: Ideal functionality for the arithmetic black box over a finite field $\mathbb{F}$.

However, this UC security model for SPDZ has properties that complicate the analysis of a practical SPDZ implementation:

**Sequential vs Concurrent Implementations.** The UC framework is used to analyze protocols in a *sequential* execution model, where the individual protocol steps (defined as interactive Turing machines) are evaluated in sequence. Consequently, no security guarantees whatsoever can be derived from the SPDZ UC security proof for a multi-threaded SPDZ implementation. In particular, the core issue here is

with a SPDZ implementation that runs several invocations of the **Output** command of $\mathcal{F}_{\mathsf{ABB}}$ in parallel, which contradicts the ideal functionality that only allows sequential calls to its commands.

We remark that what UC calls "concurrent security" is not to be confused with concurrency in the sense of multi-threading: The concurrent security guarantees of UC are about independent protocols being executed concurrently, i.e. closer to concurrency in the distributed systems sense.

**$\mathcal{F}_{\mathsf{ABB}}$, Aborts and Delayed Output.** The second property is more subtle, and concerns the way the **Output** command is defined for $\mathcal{F}_{\mathsf{ABB}}$. Since fairness cannot be guaranteed in the dishonest majority case for MPC [34], the adversary is given the choice to either let all honest parties learn the output $y$ or abort the execution (step 2)). Moreover, the adversary can make this decision *after* seeing $y$ (step 1)), and $\mathcal{F}_{\mathsf{ABB}}$ will *wait* for the adversary to communicate this decision (step 2)). This part models the capabilities of a rushing adversary. At the same time, $\mathcal{F}_{\mathsf{ABB}}$ is oblivious to the existence of MACs as those are considered a specific choice of a particular MPC protocol. As a result, $\mathcal{F}_{\mathsf{ABB}}$ hides the MAC leakage as implementation detail that is not relevant to expressing the captured security properties.

In a sequential execution, this behaviour does not cause any harm: The execution simply waits for the adversary's decision. Nevertheless, this is a previously unknown leakage vector, even in the sequential case. As we will see in the next section, the MAC key leakage has much more severe consequences when two threads execute MAC checks concurrently.

# 4. Security of SPDZ Implementations

We now discuss the architecture of existing SPDZ implementations, and analyze their susceptibility to the MAC key leakage attack from Section 3. We consider the MPC frameworks MP-SPDZ [5], SCALE-MAMBA [13] and FRESCO [14]; of all the frameworks surveyed by Hastings, Hemenway, Noble and Zdancewic [35][5], these are the only ones to implement SPDZ. We note that, as of April 2025, MP-SPDZ enjoy frequent updates and new features, FRESCO sees occasional maintenance updates, while SCALE-MAMBA has not been maintained since 2022.

## 4.1. Attack Template

The MAC key leakage attacks we found follow the same template: We assume that the attacker knows how the user specified the computation (typically this is not considered secret). The attacker will then try to replace their own local honest code with a malicious version that extracts the global SPDZ MAC key in one thread and uses it in another thread. That means that for the attack to work, it must be possible to execute two concurrent MAC checks.

5. Including later frameworks added at https://github.com/MPC-SoK/frameworks/wiki/

## 4.2. MP-SPDZ and SCALE-MAMBA

MP-SPDZ[6] and SCALE-MAMBA[7] are two forks of the discontinued SPDZ software of Keller, Scholl and Smart [12]. Both frameworks include a separate frontend for expressing the high-level computations that will be run in MPC, together with various backends that implement different MPC protocols, including SPDZ and other protocols such as those based on garbled circuits or Shamir secret sharing. For MP-SPDZ, the attacks described in this work only apply to unpatched versions with commit ID `e08a6ad` and earlier. Development of SCALE-MAMBA has now been discontinue and it is still unpatched.

The frontend allows a user to specify a program using a subset of Python[8], which will then be translated into an optimized circuit-like format, and run by a backend written in C++. The backend is essentially a virtual MPC machine that executes the relevant parts of the MPC protocol. After specifying and compiling the Python program, the user can typically later choose to run the same program under a number of different backends or with different configurations for numbers of parties, to compare performance.

The circuit-like format that is interpreted by the backend is a bytecode representation of the program, where the basic instructions are either local operations that perform some computation on locally stored registers, or interactive operations that require communication between the parties. The bytecode also contains a number of special instructions for control flow, memory access and stopping/starting threads.

**Parallelism in MP-SPDZ and SCALE-MAMBA.** The bytecode format allows expressing parallism on two levels: batching openings of secret values, and multithreading.

The first case is handled by reordering instructions: the compiler attempts to locate any instructions for opening or secret multiplication that can be executed independently, and merges them together into the same round of communication. This ensures that when evaluating a circuit, the number of communication rounds is proportional to the depth of the circuit, without requiring the programmer to manually specify each circuit layer. We note that this optimization already deviates from the formal UC specification of the arithmetic black box that SPDZ implements, where each multiplication is supposed to be carried out sequentially. However, this deviation does not lead to any vulnerabilities, since the multiplication subprotocol cannot leak sensitive information as long as the triples are not reused.

The second type of parallelism is multithreading. This form of parallelism has to be specified by the user when writing the Python program, and is enabled by exposing special functions for running instructions in a multi-threaded manner. Each thread in a program to access to a local set of registers, assigned for clear or secret data types, for storing the inputs and outputs of intructions in that thread. To

6. https://github.com/data61/MP-SPDZ
7. https://github.com/KULeuven-COSIC/SCALE-MAMBA
8. SCALE-MAMBA additionally features a Rust frontend.

facilitate complex data structures and allow communication between threads, there is a *global memory* structure that stores additional secret and clear values, which can be accessed by all threads. There are few restrictions on how threads access memory, so thread-safety must be ensured by the programmer. Each thread is also assigned its own pool of multiplication triples and other preprocessing data; however, all of this data is generated via one execution of the preprocessing protocol, so all threads share the same MAC key.

Inside each thread, MAC checks are performed before secret values are revealed. A MAC check is also performed at the end of a computation thread, and in a longer-running computation when sufficiently many secret values have been partially opened (to reduce memory footprint). Prior to our reporting of the vulnerability from Section 3, the MAC checks within each thread in MP-SPDZ were performed completely independently from each other.

**4.2.1. Attack implementation in MP-SPDZ.** To carry out the attack in MP-SPDZ, we consider the simple, example computation from Section 3.2, which computes $f_{mult}(x, y) = (\llbracket x \rrbracket \cdot \llbracket r \rrbracket) \cdot \llbracket y \rrbracket$, where $x$ and $y$ are private inputs from parties $P_2$ and $P_1$, while $r \in \mathbb{F}$ is a secret-shared random value.

Outputting $f_{mult}(x, y)$ should reveal whether one or more of the parties' inputs are zero, but nothing more. However, by exploiting the ability to steal the MAC key in a multi-threaded setting, a malicious party can learn the honest party's input. We demonstrate this by constructing a malicious version of an MP-SPDZ program that will be run by a corrupt party using a modified MP-SPDZ binary. We stress that the honest parties still use the unmodified MP-SPDZ binary and Python program that computes the desired function.

The honest version of the program we use, shown in Figure 3a, is specified to be multi-threaded, where one thread computes and outputs (1), while the other thread simply reveals another value input by one party. This is clearly an artificial use-case, but suffices to demonstrate our attack. The code uses MP-SPDZ's decorator syntax `@for_range_multithread()` to create a loop containing two parallel iterations, one for each thread, and then a conditional branch inside the loop to determine which thread body to run. The function `sint.get_input_from(i)` receives a private input from party $P_i$ and secret-shares it among the parties, while `sint.get_random()` retrieves a random secret-shared value from the preprocessing data.

The corresponding malicious code for the attack is shown in Figure 3b. This code relies on a modified version of the MP-SPDZ executable we created, with a custom command `x.reveal(malicious =1)`, which performs the output phase following the steps of the attack, i.e. it sends incorrect shares of $x$, never opens the commitments in the MAC check and returns the MAC key.

The key is written to memory and is used in the second thread, while the first thread hangs as $P_1$ waits for a response in the MAC check. In the second thread, the corrupt $P_2^*$ waits until the MAC key is written to memory, then changes the product `xr` by one and uses another custom function `add_to_mac()` to adjust the MAC. As a result, the MAC check in the output phase `reveal()` succeeds and both parties learn the output $y$, $P_1$'s secret input.

**4.2.2. Attack in SCALE-MAMBA.** SCALE-MAMBA shares a similar structure and has the same multi-threading capabilities as MP-SPDZ. It is therefore also vulnerable to the same exploit, and has not been patched as SCALE-MAMBA is no longer maintained. However, we point out that, due to other implementation issues in SCALE-MAMBA, there are even easier ways to bypass the MAC check without relying on multi-threading. We refer to Section 6 for details.

**4.2.3. Broader impact of the attack.** Although the attack demonstration above is for a rather contrived program, we believe that vulnerable instances could easily occur in real use-cases. Firstly, we note that multi-threading is commonly used to optimize MPC implementations, and for the case of MP-SPDZ, several recent prototype applications have relied on multi-threading in some form [36], [37], [38]. If any of these applications perform a MAC check during a multi-threaded component of the program, the correctness and privacy guarantees of data processed in other threads could be compromised. A natural scenario where this may occur frequently is a client-server MPC setting, where clients make queries to a set of MPC servers holding secret-shared data, and each client is handled by a separate thread. Without the appropriate countermeasures, this becomes insecure when using SPDZ, since each client thread will require at least one MAC check before sending the client's output.

## 4.3. FRESCO

FRESCO is a Java-based framework for MPC. Unlike the previous frameworks, FRESCO does not provide a high-level frontend, but instead requires the programmer to write the computation directly in Java. Depending on whether the computation is Boolean or arithmetic, FRESCO allows it to be run with several different protocols, including SPDZ [28], SPDZ2k [32], MASCOT [29] and a passively secure version of the TinyTable protocol [39].

FRESCO also supports user-specified multi-threading, however, in a much more restricted manner than MP-SPDZ and SCALE-MAMBA. It provides some constructs that allow for instructions to be parallelized, however, this does not allow for *output* instructions to be run in parallel.

**4.3.1. Susceptibility to the attack.** Since FRESCO does not allow for parallelization of output instructions, the MAC check is never run concurrently so seems to be unaffected by leakage of the key during an invalid MAC check. We note, however, that we identified another issue in the FRESCO MAC check that can be exploited, in Section 6.

```
@for_range_multithread(2, 2, 2)
def _(i):
  y = sint.get_input_from(0)
  x = sint.get_input_from(1)
  r = sint.get_random()
  @if_e(i==0)
  def _(): # Thread 0 body

    res = x.reveal()
    print_ln('Result: %s', res)

  @else_
  def _(): # Thread 1 body



    xr = x*r



    res = y * xr
    print_ln('Result: %s',
             res.reveal())
```

(a) Honest code run by party $P_1$

```
alpha_mem = memorize(cint(0))
@for_range_multithread(2, 2, 2)
def _(i):
  y = sint.get_input_from(0)
  x = sint.get_input_from(1)
  r = sint.get_random()
  @if_e(i==0)
  def _(): # Thread 0 body
    alpha = x.reveal(malicious=1)
    print_ln('MAC Key: %s', alpha)
    alpha_mem.write(alpha)
    wait(30)
  @else_
  def _(): # Thread 1 body
    @do_while
    def _():
      wait(1)
      return alpha_mem.read() == 0
    xr = x*r
    xr += 1
    add_to_mac(xr,
               -alpha_mem.read())
    res = y * xr
    print_ln('y = %s',
             res.reveal())
```

(b) Malicious code run by party $P_2^*$

Figure 3: MP-SPDZ programs to demonstrate a toy example of the MAC key leakage attack. The honest program computes $x \cdot r \cdot y$ in thread 1, while revealing an arbitrary value in thread 0. The corrupt $P_2^*$ learns the MAC key in thread 0 and exploits it in thread 1.

## 5. Mitigating MAC Key Leakage Attacks

The attack as described relies on being able to extract the MAC key in one thread by maliciously performing only part of the MAC check, and then using the key in another thread. Countermeasures will either eliminate the second thread entirely, eliminate the MAC key leakage altogether or try to mitigate its effect by additional thread synchronization. We group these countermeasures into two categories: ones which completely avoid any MAC key leakage, and ones that only aim to mitigate its effect.

### 5.1. Synchronization Mechanisms

Perhaps the simplest solution would be to revert back to a sequential implementation, removing all multi-threading or concurrent execution from the protocol. Unfortunately, this sacrifice of efficiency may not be suitable in practical applications. Another option is to keep the protocol as is, but carefully examine it again to understand which parts can (probably) safely be parallelized, and where additional synchronization is needed. In this case, the protocol still has the potential to leak the MAC key, the implementation just prevents attacks based on this leakage.

We consider two different approaches to implementing the necessary synchronization, which have been used in MP-SPDZ and FRESCO, and discuss their performance impact.

**5.1.1. Lock/mutex.** MP-SPDZ solved the MAC check leakage issue (in commits 6a42453 and b86f29b) by requiring that some computations of concurrent threads be done sequentially. The following code, which corresponds to step 2 and 3 of the MAC check protocol (Figure 1), is executed when a MAC check is called.

```
(...)
P.Broadcast_Receive(Comm_data);
coordinator.wait(P.get_id());
P.Broadcast_Receive(Open_data);
(... check commitment openings ...)
coordinator.finished();
(...)
```

A thread exchanges the commitment data, then acquires a shared lock (handled by the coordinator), then exchanges the commitment opening data, checks the openings and then releases the lock. Only one thread is allowed this lock at a time. As a consequence, if one thread is not receiving any opening data, then no other thread can complete a MAC check. In our attack, one thread needs to extract the MAC key but never send the opening data, while another thread continues computation, so using this lock prevents the attack.

**5.1.2. Global MAC Check Thread.** Along the same lines as the previous solution is the idea of performing all the MAC checks in a main thread as in FRESCO. Specifically, when a thread wants to perform a check, it will wait for all threads to synchronize and perform the MAC check in a single thread with all the unchecked partially opened

values of all the different threads. Clearly, with synchronized checks the described attack is no longer realisable. We note that synchronizing threads is bound to have some negative impact on the throughput of each individual thread.

## 5.2. Protocol Modifications

Another option is to modify the *protocol* to remove the leaky MAC check entirely, ruling out the possibility of any implementation bugs that lead to exploits due to MAC key leakage. This direction follows in the philosophy of misuse-resistant cryptography, originally proposed in the context of nonce reuse resistance for authenticated encryption [40], where protocols are designed to be more resilient to implementation errors. Furthermore, it obtains the additional benefits of avoiding any potential performance loss due to thread synchronization, and avoiding the need to re-generate unused preprocessing material in case the protocol aborts.

**5.2.1. SPDZ with BDOZ-style MACs.** BDOZ [23] is an MPC protocol that can be seen as the predecessor of SPDZ. While the SPDZ online phase uses a global MAC key to authenticate shares, the BDOZ protocol authenticates each share separately using pairwise MACs, as described in Section 2.2. When opening a secret shared value $x$, a party $P_i$ sends share $x_i$ and MAC $t_{ij}^{(x)}$ to party $P_j$. $P_j$, holding MAC keypair $(\alpha_j, \beta_{ji}^{(x)})$, will verify that

$$t_{ij}^{(x)} = \alpha_j \cdot x_i + \beta_{ji}^{(x)}$$

Note that, when opening a secret value and performing the MAC check, no information about the keypair is leaked to the other parties, except for information about whether the MAC check succeeded or not. As a result, the pairwise MAC representation eliminates attack vectors that exploit leaking keys during the MAC check protocol.

**5.2.2. SPDZ with Masked MAC check.** Another option is to use a masked version of the SPDZ MAC check protcol. Remember what goes wrong in the SPDZ MAC check protocol in Figure 1: Consider shares $(\sigma_1, \ldots, \sigma_n)$, where $\sigma_i = \gamma_i^{(x)} - x \cdot \alpha_i$. The parties want to check that $\sigma := \sum_i \sigma_i = 0$. However, if some party $P_i$ broadcasts an incorrect share $x_i' \neq x_i$ when opening $x$, the value $\sigma$ reveals the global MAC key $\alpha$.

To fix this, we propose to randomize the shares of $\sigma$ before the commit-and-open protocol. This can be done by taking a preprocessed multiplication triple $(a, b, c)$, and using it to multiply $\sigma$ with the random mask $b$. Note that this multiplication does not need to be authenticated, as $\sigma$ is not authenticated, and any cheating in the multiplication protocol will simply lead to an abort. The revised MAC check protocol is shown in Figure 4.

In the following lemma, we analyze the correctness and security properties of the standalone masked MAC check. In Appendix B, we further provide a UC security analysis of SPDZ in a concurrent setting, showing that the masked MAC check allows a multi-threaded online phase where each thread shares the same MAC key.

**Protocol** $\Pi_{\mathsf{MaskedCheck}}(x, [\![x]\!])$

1) Party $P_i$ parses its share of $[\![x]\!]$ as $(x_i, \gamma_i, \alpha_i)$
2) The parties take shares of a random, preprocessed unauthenticated multiplication triple, $(a_i, b_i, c_i)$
3) Each party $P_i$ computes $\sigma_i := x\alpha_i - \gamma_i - a_i$
4) Each party broadcasts $\sigma_i$ and reconstructs $\sigma = \sum_i \sigma_i$
5) Each party $P_i$ computes $z_i = \sigma \cdot b_i + c_i$
6) Each party commits to $z_i$ to all other parties.
7) After receiving all other parties' commitments, each party opens their commitment to reveal $z_i$
8) If any opening fails, abort.
9) If $\sum_i z_i = 0$, output success, otherwise abort.

Figure 4: Masked version of the SPDZ MAC check

**Lemma 1.** *The protocol* $\Pi_{\mathsf{MaskedCheck}}$ *is: (1)* correct*, that is, if* $x$ *was opened correctly and all parties follow the protocol, the output succeeds; (2)* secure*, that is, if the opened input is* $x' \neq x$*, then the honest parties abort with probability at least* $1 - 1/|\mathbb{F}|$*; and (3)* private*, meaning the honest parties'* $\sigma_i, z_i$ *messages seen by any malicious adversary can be simulated given only the output* $x$ *and the shares of the corrupted parties.*

*Proof.* Suppose the parties have opened a value $x' = x + e$, for some possible error $e$. Suppose also that each $P_i$ sends the value $\sigma_i' = \sigma_i + f_i$ in step 4 (where corrupt parties may choose non-zero errors $f_i$), reconstructing $\sigma' = \sum_i \sigma_i' = x'\alpha - x\alpha - a + \sum_i f_i = e \cdot \alpha + \sum_i f_i$. We have:

$$\sum_i z_i = \sigma' \cdot b + c = (e\alpha - a + \sum_i f_i) \cdot b + c = e \cdot \alpha \cdot b + \sum_i f_i \cdot b$$

In the case where all parties behave honestly and the $e, f_i$ errors are zero, then the above equals zero so the check will always pass. This shows the correctness property.

For security, suppose that $e \neq 0$. If an adversary manages to find a set of corrupt $z_i$ shares where the check passes, then by rearranging the above equation to solve for $\alpha$, we see that this can only happen with probability $1/|\mathbb{F}|$. Finally, for privacy, we argue that the honest parties' opened shares $\sigma_i$ and $z_i$ leak no information about the MAC key. In particular, each $\sigma_i$ is perfectly masked by the fresh random share $a_i$ from the preprocessing phase, so can be perfectly simulated by a uniform field element. If the output $x$ and corrupted parties' shares $z_i$ were computed honestly, then the honest $z_i$ shares can be simulated by random field elements such that $\sum_{i=1}^{n} z_i = 0$. If $x$ was opened incorrectly, or if any $z_i$ is incorrect, then the honest $z_i$'s can be simulated uniformly at random. In both cases, the simulated shares are within statistical distance $1/|\mathbb{F}|$ of uniform, because as long as

$b \neq 0$ then they are uniform shares of a random, non-zero value, due to the randomness of $b$ and the $c_i$'s. □

## 5.3. Performance Impact

We have described two main mitigation strategies, synchronization at the implementation level and protocol modifications. We will now compare their impact on performance and make recommendations.

**5.3.1. Lock vs. Global MAC Check Thread.** Using the lock, if there are $k$ concurrent threads that each perform a MAC check, all $k$ checks will have to executed sequentially. This adds $k$ rounds of interaction, compared with the (insecure) approach of running all MAC checks in parallel. On the other hand, with the solution of a global MAC check thread and performing all checks in one thread, the round complexity does not increase, but there may be some additional overhead from passing all unchecked openings from each thread into the main thread. Therefore, it seems that the lock-based countermeasure will be preferable in settings with very low network latency, while performing all checks in one thread is better-suited to scenarios where latency is high and each thread is not handling large amounts of data between MAC checks.

Note that any additional synchronization incurs a slight performance loss and relies on the implementers' ability to assess the parallelizability appropriately as well as securely implement the chosen synchronization mechanism. Another drawback is in case of an aborting execution, the protocol execution cannot be restarted with the remaining correlated randomness. Instead, all stored correlated randomness has to be discarded and a new preprocessing phase *must* be run to generate fresh correlated randomness. As we will see in Section 6.2.4, the latter is a new source of implementation errors.

**5.3.2. BDOZ-Style MAC vs. Masked MAC Check.** With BDOZ-style MACs, there is the overhead that the authentication of one value requires each party $P_i$ to store a MAC and local key for every other party. Compared with SPDZ, where each party has just one field element, the storage and local computation costs during circuit evaluation increase by a factor $2(n-1)$ for $n$ parties. With the masked MAC check, on the other hand, we retain the storage and computational efficiency of the compact, SPDZ MACs. The downside is that the MAC check protocol now requires 3 rounds of interaction instead of 2, and also an extra (unauthenticated) multiplication triple. The latter is slightly more efficient than generating the normal authenticated triples, so is unlikely to be a bottleneck, except perhaps in applications with a very large number of MAC checks. In general, BDOZ-style MACs may be preferable with a small number of parties due to their simpler MAC check protocol, while the masked SPDZ MAC check will be more efficient for many-party settings.

However, the increased round complexity is still much better than the factor $k$ increase from the lock-based solution (5.1.1) with $k$ threads. Overall, we conclude that the masked MAC check seems preferable in scenarios where parallelism is critical for performance.

**5.3.3. Synchronization vs. Protocol Modification.** Independently of performance overheads, we note that when modifying the protocol to use either BDOZ or the masked SPDZ check, we obtain the qualitative benefits of allowing unrestricted multi-threading during MAC checks, as well as not having to discard correlated randomness after an abort.

When considering the performance of the different approaches, all the mitigations have some overhead on top of SPDZ. In many applications, however, MAC checks only need to be performed relatively infrequently, so the additional costs would be negligible compared with the overall running time of the protocol. Nevertheless, there are certain use-cases that might see a significant performance degradation:

1) Settings with highly *reactive* computations, which repeatedly reveal the output of many, small computation tasks. For example, this could arise in an outsourced MPC setting, where several MPC servers are performing requests on behalf of a large number of clients [41]. Here, it would be natural to handle each request in a separate thread, so that one client's response does not depend on another client who may have a slower connection. However, using a lock or global MAC check thread essentially means all clients would have to be handled at once.
2) As we will discuss in Section 6.2.3, certain truncation and comparison protocols involve masked opening of a secret input, and to avoid information leakage, require all previous openings to be checked beforehand. This implies that, in a use-case where a large number of comparisons are to be performed, MAC checks must be carried out between every round of comparisons. When using synchronization-based countermeasures, this limits the use of multi-threading to speed up the comparisons, since every MAC check would require a synchronization step across the threads.

Ultimately, the cost of each strategy depends on the specific application and the type of computation that is performed.

# 6. Further Pitfalls and Vulnerabilities

We discuss some additional pitfalls when implementing SPDZ and related protocols, including some vulnerabilities we have identified in SCALE-MAMBA, MP-SPDZ and FRESCO.

## 6.1. Thread Memory Transfer Attack

SPDZ implementations such as MP-SPDZ and SCALE-MAMBA allow threads to exchange data using shared memory, including the transfer of secret-shared values between different threads. This raises the question of when and

how to perform MAC checks for shared secret values. We show that, by taking advantage of missing MAC checks resulting from issues with the implementation of global thread memory, parties can be forced to open incorrect values that may leak on private inputs. We will first describe the underlying idea of the attack and then present a concrete implementation on MP-SPDZ.

**6.1.1. Attack Description.** Consider a multi-threaded SPDZ implementation where secret values (i.e. shares and their MACs) are passed from one thread to another via global memory. To understand the attack, we first need to discuss what options exist for handling MAC checks.

Consider a computation using two threads $T_1, T_2$ where a secret-shared value $[\![x]\!]$ is transferred from $T_1$ to $T_2$ via the global memory. Let $\mathsf{Auth}_x$ be the set of opened values, and their corresponding MAC shares, resulting from the computation of $[\![x]\!]$, restricted to those whose MACs have not yet been checked. We note that the MACs in $\mathsf{Auth}_x$ must be checked before $[\![x]\!]$ can be safely opened. In principle, there are different options for where $\mathsf{Auth}_x$ should be checked:

1) $\mathsf{Auth}_x$ is only checked in $T_1$. $[\![x]\!]$ is passed to $T_2$ and when the output phase is entered in thread $T_1$, the MACs in $\mathsf{Auth}_x$ are checked.
2) $\mathsf{Auth}_x$ is only checked in $T_2$. When $[\![x]\!]$ is sent from $T_1$ to $T_2$, the relevant authentication data $\mathsf{Auth}_x$ is also transferred to $T_2$ and checked when a MAC check is performed in $T_2$.
3) $\mathsf{Auth}_x$ is checked in both $T_1$ and $T_2$.

The last option is clearly secure: the added redundancy ensures that a value $[\![x]\!]$ is never opened until its unchecked authentication set $\mathsf{Auth}_x$ is empty. We note that performing the same MAC check multiple times is allowed since honestly performed MAC checks leak no information and any deviation from the protocol is detected with overwhelming probability.

The remaining options 1 or 2 are unfortunately not sufficient to guarantee security in all contexts. It is not hard to see that a malicious adversary can modify shares before multiplications, and remain undetected as long as the secret-shared value (or the relevant authentication data) is sent to another thread. Option 1 is clearly insecure since $[\![x]\!]$ might be opened in thread $T_2$ before $\mathsf{Auth}_x$ is checked in $T_1$, which means that an incorrect $x$ (that may depend on sensitive inputs) will be opened. Option 2, where $\mathsf{Auth}x$ is passed to $T_2$ for checking along with $[\![x]\!]$, is also insecure. Even if thread $T_1$ does not use $[\![x]\!]$ after sending it to $T_2$, there may be another secret-shared value $[\![y]\!]$ in $T_1$ with authentication data $\mathsf{Auth}_y$ that intersects with $\mathsf{Auth}_x$. Now that responsibility for the MACs in $\mathsf{Auth}_x$ have been passed over to $T_2$, the value $y$ cannot be fully verified in thread $T_1$ since some of $\mathsf{Auth}_y$ is no longer held by $T_2$.

**6.1.2. Implementations.** It turns out that several implementations had implemented the sharing of secret values via global memory in the style of Option 1, and thus they were vulnerable to our attack.

**MP-SPDZ.** To transfer secret-shared variables between threads, MP-SPDZ opted for Option 1, where partially opened values are checked in the thread in which they are created.[9] We therefore present code that illustrates the attack.

We will use the running example from Section 3.2 again, where the honest parties wish to compute product $x \cdot r \cdot y$. Here, we require that the result is opened in a different thread to the one where the shares are created. In the honest program in Figure 5a, the result is written the global memory, while the second thread periodically polls the memory until it can read the result and open it.

To break this protocol, the malicious $P_2^*$ locally runs the attack code in Figure 5b. As expected, the corrupt party $P_2^*$, with input 0, can modify the result and learn $P_1$'s input because the maliciously chosen shares are checked (too late) in another thread.

**SCALE-MAMBA.** Like the previous attack, this vulnerability also applies to SCALE-MAMBA. Again, implementing the attack on this framework is of less interest due to the additional vulnerability of SCALE-MAMBA, discussed later in this section.

**FRESCO.** FRESCO does not support transferring of secret values between threads and is thus not vulnerable to this attack.

## 6.2. Additional Pitfalls

We describe a number of additional pitfalls that can cause serious vulnerabilities when implementing SPDZ. While some of these are rather basic errors, they were each present in at least one of the frameworks we studied, so we believe it may be useful to catalogue them to help understand the kinds of things that can go wrong.

**6.2.1. Insecure hash-based commitments.** A textbook example of a commitment scheme is to compute $c = H(m\|r)$ to commit to a message $m$ with randomness $r$, where $H$ is a secure cryptographic hash function. This scheme is well-known to be a binding commitment if $H$ is collision-resistant, and hiding if $\{H(\cdot\|k)\}_k$ is a pseudorandom function family. However, this construction is in general *not composable* and unsuitable for use in the SPDZ MAC check. In particular, in an interactive setting the scheme is vulnerable to a replay attack, where an attacker reuses a commitment $c$ that was sent by another party.

We point out that this does not contradict the folklore wisdom that the construction $H(m\|r)$ can in fact be proven UC secure (see e.g. [28, Appendix A]), albeit in a very strong ideal model where commitments are created using a "local" random oracle $H$ that is independently sampled for each new commitment. While this model is often used in UC security analysis [42], when instantiating $H$ with

9. As with the previous attack, this was changed after reporting the issue, and MP-SPDZ is no longer vulnerable.

```
res_mem = memorize(sint(0))
go_flag = memorize(regint(0))
@for_range_multithread(2, 2, 2)
def _(i):
  @if_e(i==0)
  def _(): # Thread 0 body
    y = sint.get_input_from(0)
    x = sint.get_input_from(1)
    r = sint.get_random()
    xr = x * r

    res = xr * y
    res_mem.write(res) # Write to global memory
    go_flag.write(regint(1))

  @else_
  def _(): # Thread 1 body
    @do_while
    def _():
      wait(1)
      return (go_flag.read() == 0)
    print_ln('Result: %s',
             res_mem.read()
                    .reveal())
```

(a) Honest code run by party $P_1$

```
res_mem = memorize(sint(0))
go_flag = memorize(regint(0))
@for_range_multithread(2, 2, 2)
def _(i):
  @if_e(i==0)
  def _(): # Thread 0 body
    y = sint.get_input_from(0)
    x = sint.get_input_from(1)
    r = sint.get_random()
    xr = x * r
    xr += 1 # Tamper with xr
    res = xr * y
    res_mem.write(res)
    go_flag.write(regint(1))
    wait(30)
  @else_
  def _(): # Thread 1 body
    @do_while
    def _():
      wait(1)
      return (go_flag.read() == 0)
    print_ln('Result: %s',
             res_mem.read()
                    .reveal())
```

(b) Malicious code run by party $P_2^*$

Figure 5: MP-SPDZ programs for the unchecked thread memory attack. The honest protocol intends to compute the product $x \cdot r \cdot y$ in thread 0, and then open the result in thread 1. The corrupt $P_2^*$ deviates and stalls thread 0 long enough to learn the honest party's input.

a concrete hash function, one should always take care to include domain separation information such as session and party identifiers as input to the hash.

In SPDZ, implementations that do not properly use domain separation allow for attacking the "commit-and-open" phases of the MAC check in two possible ways:

1) In the single MAC check protocol (Figure 1), a rushing adversary can wait to receive a commitment $c = H(\sigma_1 \| r_1)$ from the honest party $P_1$, and then send the same $c$ as its own commitment, and then later send the honest $P_1$'s opening information $(\sigma_1, r_1)$. In the two-party setting with one corruption, the MAC check will pass when working in a field of characteristic two, since $\sigma_1 + \sigma_1 = 0$.
2) The batch MAC check protocol (Figure 8 in Appendix A) runs a coin-tossing protocol to generate a seed used to produce a set of random field elements that define a linear combination.
   A common way to implement coin-tossing is for each party to commit to a random seed $s_i$, then open all commitments and output the randomness $s_1 \oplus \ldots s_n$. Unfortunately, when committing with $H(s_i \| r_i)$, the same replay attack allows a corrupt party to cancel out the seed of one honest party, by reusing their commitment. If there are at least $n/2$ corruptions then all honest parties' contributions can be cancelled, and the seed is completely predictable to the adversary. As discussed in Appendix A, this makes it straightforward to cheat in the batch MAC check.

Of the three implementations we surveyed, FRESCO is the only one that uses $H(m \| r)$ as a commitment scheme. Since FRESCO does not implement SPDZ over binary fields, the first attack is not applicable. However, it does implement batch MAC checking and coin tossing with the approach described above, which makes it vulnerable to the second attack.

MP-SPDZ and SCALE-MAMBA bypass the issue by committing with $H(i \| x \| r)$, where $i$ is the index of the sending party encoded to a fixed length. This prevents a commitment from being reused by another party in this session, avoiding the attack. In general, in applications that require commitments in a concurrent setting, it is recommended to also incorporate a unique session identifier $sid$, via the commitment $H(sid \| i \| x \| r)$, although this appears to not be needed here.

**6.2.2. Missing MAC checks.** The SPDZ protocol can become insecure in case MACs are not checked at the correct points in the execution. Recall that before opening any output of the computation, the parties must be sure to check the MACs of all openings it depends on. Furthermore, the MAC on an output should be checked immediately after it is opened.

In SCALE-MAMBA, we found that the initial MAC check of the output phase is missing. When running a SPDZ MPC program using SCALE-MAMBA, the only time a MAC check is performed in the online phase is at program termination, at the end of a thread and, to save memory space, when the number of partial openings exceed a certain number (by default the threshold is set to 1,000,000). To

```
x = sint.get_private_input_from(1)
y = sint.get_private_input_from(0)
r = sint.get_random_triple()[0]

xr = x*r

res = xr*y
print_ln("Result: %s",
         res.reveal())
```

(a) Honest party $P_1$

```
x = sint.get_private_input_from(1)
y = sint.get_private_input_from(0)
r = sint.get_random_triple()[0]

xr = x*r
xr += 1
res = xr*y
print_ln("Result: %s",
         res.reveal())
```

(b) Corrupt party $P_2^*$

Figure 6: Attack on the missing MAC check in SCALE-MAMBA. The corrupt party $P_2^*$ chooses $x = 0$ and deviates from the protocol to learn the honest party's input, $y$.

demonstrate this, in Figure 6, we show a simple example of a SCALE-MAMBA MPC program where the dishonest party obtains the secret input of the honest party, using the same example as in Section 4.

The command `sint.get_secret_input_from(i)` assigns party $P_i$'s secret input and `sint.get_random_triple()[0]` generates a random secret-shared field element. Running the program outputs the secret input of $P_1$, followed by a MAC check error.

**6.2.3. More missing checks: overflow leakage in truncation protocols.** Many MPC protocols rely on the opening operation as a subroutine, in order to improve efficiency for certain higher-level applications such as truncation of secret-shared values. For example, Figure 7 shows a protocol for reducing a secret-shared value modulo 2. The protocol works under the condition that the input $x$ lies in the range $\{0, \ldots, 2^k - 1\}$. In the first line, $x$ is masked with a random value $r'$ and a random bit $r_0$; the value $c$ is within statistical distance $2^{-s}$ of uniform, because of the large range $r'$ comes from, so hides the input $x$. After opening $c$, shares of the least significant bit of $x$ can be easily obtained through local computation.

$\Pi_{\mathsf{BatchCheckMAC}}(x_1, \ldots, x_m, [\![x_1]\!], \ldots, [\![x_m]\!])$

The protocol is parameterized by a security parameter $s$ and a maximum bit-length $k$, and works over $\mathbb{F}_p$ where $p > 2^{k+s}$.

**Preprocessing:** Shares $[\![r_0]\!], [\![r']\!]$, where $r_0$ is a random bit and $r' \leftarrow \{0, \ldots, 2^{k+s}\}$.

1) Compute $[\![c]\!] = [\![x]\!] + 2[\![r']\!] + [\![r_0]\!]$
2) Open $[\![c]\!]$ to obtain $c$
3) Return $[\![x_0]\!] = (c \bmod 2) + [\![r_0]\!] - 2(c \bmod 2)[\![r_0]\!]$

Figure 7: Protocol for reduction modulo 2

This protocol "mask-and-open" protocol structure, relying on preprocessed random values in a certain range, is common to many MPC building blocks for arithmetic computations, such as secure comparison and truncation. It was pointed out in [43] that before opening $c$, it must be verified that $[\![x]\!]$ was computed correctly. Otherwise, it could be that $x$ lies outside of the range $\{0, \ldots, 2^k - 1\}$, and revealing $c$ may leak more information than desired about honest parties' inputs. In SPDZ, therefore, a MAC check protocol should always be run before opening $c$.

MP-SPDZ uses protocols with a similar structure to Figure 7 for many of its arithmetic building blocks. While most of the time, it takes care to force a MAC check before opening the masked value $c$, it turns out that the check was missing in a probabilistic truncation protocol, which could lead to undesirable leakage with malicious behaviour.

As mentioned above, SCALE-MAMBA does not typically check MACs until the end of the program and hence this vulnerability is present in all of its truncation or comparison protocols.

FRESCO also implements several protocols in this family, to help with arithmetic building blocks. FRESCO checks MACs before opening the masked values, so is not vulnerable to this issue.

**6.2.4. Using Correlated Randomness after abort.** A more subtle issue arises after an abort in the SPDZ protocol. Because of the potential for MAC key leakage as in Section 3, no correlated randomness that was previously generated under the same MAC key is safe to use after abort, even if the triples themselves have not been used.

Since the current version of MP-SPDZ starts a fresh preprocessing protocol for each execution by default, one would think this is not an issue. However, it turns out that when generating a MAC key, the preprocessing phase first checks if a MAC key has already been stored on disk, and if so uses the existing key rather than sampling a new one. Hence, the implementation is still vulnerable to the MAC key leakage attack, where an abort in the online phase will allow an adversary to cheat in any future protocol execution, unless the MAC key is manually erased by the user.

FRESCO, on the other hand, does not store MAC keys or correlated randomness for later usage, so is not vulnerable to this kind of attack.

## 7. Conclusion

In this work, we surveyed implementations of the SPDZ protocol for secure multiparty computation with malicious security: MP-SPDZ, SCALE-MAMBA and FRESCO. We discovered multiple attacks around the theme of MAC checks in combination with a rushing adversary. While MP-SPDZ and SCALE-MAMBA were both vulnerable to a surprising number of different attacks, FRESCO was only affected by a single attack (using an insecure commitment scheme that is widely (and falsely) believed to be UC-secure in the multiparty case). Most of the vulnerabilities have been disclosed to the developers of the respective frameworks, and MP-SPDZ has been patched; some of the additional vulnerabilities we found have not yet been disclosed, but we are following standard practices of informing developers and allowing sufficient time to fix them before making this work public.

Despite the focus of this work on the alarming number of uncovered flaws, we also want to add a more positive interpretation: MPC is still at a very early stage of adoption, and we hope that our findings can inform the development of future MPC protocols and implementations. For one, our work can serve as starting point for developing best practices and collect known pitfalls specific to MPC. Moreover, because of the inherent complexity of MPC protocols, let alone their implementations, we believe that it might make sense to focus more on developing misuse-resistant MPC protocols that are more resilient to implementation flaws. As a concrete example, consider the interplay between subtle aspects of the SPDZ protocol and the implementation security: The MAC key leakage property we identified is an unusual property that does not cause issues in a sequential execution, but is devastating in the concurrent setting unless complicated synchronization mechanism are in place. On the theory side, the identified gaps between existing security models and the reality of implementations can serve as a starting point for further exploration of more fine-grained security models that are suitable to capture these subtle nuances in security.

### 7.1. Recommendations

**... for MPC researchers.** Academic researchers should be more explicit about clarifying implicit assumptions. In this case, highlighting which parts of a protocol (if any) can be safely parallelized without violating the security model, and how to securely handle aborts. They should also be aware of the potential of leaky consistency checks such as the one identified in Section 3.

**... for MPC developers and their users.** Developers should follow best practices for developing cryptographic code in general, paying particular attention to the risks of concurrency and handling aborts. In particular, after any abort potentially caused by cheating, all randomness used in the current execution should be erased, and any new execution must begin with fresh randomness, which includes correlated randomness from a preprocessing phase[10]. Ideally we would also encourage anyone to not deviate from protocol specifications without a rigorous understanding of the implications. Unfortunately, this is not always practical — as is the case with UC modelling that does not even cover basic multi-threading optimizations — so all we can do in the meantime is to advise to proceed with care. Finally, we remark that none of the surveyed implementations included a prominent disclaimer about the prototype status on their github repository at the time of writing (though to be fair, all three implementations mention the prototype status at some point in their README file or documentation). Given the amount of industry attention MPC receives at the moment, we feel that such a disclaimer would be warranted.

### 7.2. Future Work

Apart from new security models and proof techniques to analyse the effects of parallelization, we can see potential for future tool support: It might be possible to support users in choosing appropriate parallelization levels with the help of formal tools that detect, for example, secret-dependent parts of the computation. A long term goal could be to develop compilers that automate the parallelization of MPC implementations. A similar approach has been proposed for zero-knowledge proofs [44]. The approach relies on existing theoretical insights about the security of parallel zero-knowledge proof instances with shared witness. In the case of MPC, developing a theory of what can securely be parallelized is an interesting open problem.

## Acknowledgments

The authors would like to thank the maintainers of the affected MPC frameworks, in particular Marcel Keller, Danil Annenkov and Peter Frandsen, for their collaboration in fixing the identified issues. The authors also thank the anonymous S&P reviewers for their insightful comments, and the attendees of the TPMPC 2024 and RWMPC 2025 workshops for helpful discussions about challenges around MPC implementations.

Sabine Oechsner was supported by the Blockchain Technology Laboratory at the University of Edinburgh and Input Output Global. Peter Scholl was supported by the Danish Independent Research Council under Grant-ID DFF-0165-00107B (C3PO) and by the Digital Research Center Denmark (DIREC).

## References

[1] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *2017 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2017, pp. 19–38.

10. These practices are discussed at https://www.coinbase.com/blog/the-subtleties-of-error-handling-flaws-in-mpc in the context of threshold ECDSA implementation bugs.

[2] P. Bogetoft, D. L. Christensen, I. Damgård, M. Geisler, T. Jakobsen, M. Krøigaard, J. D. Nielsen, J. B. Nielsen, K. Nielsen, J. Pagter, M. I. Schwartzbach, and T. Toft, "Secure multiparty computation goes live," in *FC 2009: 13th International Conference on Financial Cryptography and Data Security*, ser. Lecture Notes in Computer Science, R. Dingledine and P. Golle, Eds., vol. 5628. Springer, Berlin, Heidelberg, Feb. 2009, pp. 325–343.

[3] D. W. Archer, D. Bogdanov, Y. Lindell, L. Kamm, K. Nielsen, J. I. Pagter, N. P. Smart, and R. N. Wright, "From keys to databasesreal-world applications of secure multi-party computation," *The Computer Journal*, vol. 61, no. 12, pp. 1749–1771, 2018.

[4] D. Malkhi, N. Nisan, B. Pinkas, and Y. Sella, "Fairplay - secure two-party computation system," in *USENIX Security 2004: 13th USENIX Security Symposium*, M. Blaze, Ed. USENIX Association, Aug. 2004, pp. 287–302.

[5] M. Keller, "MP-SPDZ: A versatile framework for multi-party computation," in *ACM CCS 2020: 27th Conference on Computer and Communications Security*, J. Ligatti, X. Ou, J. Katz, and G. Vigna, Eds. ACM Press, Nov. 2020, pp. 1575–1590.

[6] I. Levi and C. Hazay, "Garbled circuits from an SCA perspective: Free XOR can be quite expensive. . ." *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 2, pp. 54–79, 2023.

[7] M. Hashemi, D. Forte, and F. Ganji, "Time is money, friend! Timing side-channel attack against garbled circuit constructions," in *ACNS 24: 22nd International Conference on Applied Cryptography and Network Security, Part III*, ser. Lecture Notes in Computer Science, C. Pöpper and L. Batina, Eds., vol. 14585. Springer, Cham, Mar. 2024, pp. 325–354.

[8] M. Hashemi, D. M. Mehta, K. Mitard, S. Tajik, and F. Ganji, "Faulty-garble: Fault attack on secure multiparty neural network inference," in *Workshop on Fault Detection and Tolerance in Cryptography, FDTC 2024, Halifax, NS, Canada, September 4, 2024*. IEEE, 2024, pp. 53–64.

[9] T. Haines, S. J. Lewis, O. Pereira, and V. Teague, "How not to prove your election outcome," in *2020 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2020, pp. 644–660.

[10] Q. Dao, J. Miller, O. Wright, and P. Grubbs, "Weak fiat-shamir attacks on modern proof systems," in *2023 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2023, pp. 199–216.

[11] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Advances in Cryptology – CRYPTO 2012*, ser. Lecture Notes in Computer Science, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, Berlin, Heidelberg, Aug. 2012, pp. 643–662.

[12] M. Keller, P. Scholl, and N. P. Smart, "An architecture for practical actively secure MPC with dishonest majority," in *ACM CCS 2013: 20th Conference on Computer and Communications Security*, A.-R. Sadeghi, V. D. Gligor, and M. Yung, Eds. ACM Press, Nov. 2013, pp. 549–560.

[13] A. Aly, K. Cong, D. Cozzo, M. Keller, E. Orsini, D. Rotaru, O. Scherer, P. Scholl, N. P. Smart, T. Tanguy *et al.*, "Scale–mamba v1. 14: Documentation," *Documentation. pdf*, 2021.

[14] Alexandra Institute, "FRESCO - a FRamework for Efficient Secure COmputation," https://github.com/aicis/fresco.

[15] V. Kolesnikov and T. Schneider, "Improved garbled circuit: Free XOR gates and applications," in *ICALP 2008: 35th International Colloquium on Automata, Languages and Programming, Part II*, ser. Lecture Notes in Computer Science, L. Aceto, I. Damgård, L. A. Goldberg, M. M. Halldórsson, A. Ingólfsdóttir, and I. Walukiewicz, Eds., vol. 5126. Springer, Berlin, Heidelberg, Jul. 2008, pp. 486–498.

[16] A. C.-C. Yao, "Protocols for secure computations (extended abstract)," in *23rd Annual Symposium on Foundations of Computer Science*. IEEE Computer Society Press, Nov. 1982, pp. 160–164.

[17] S. Shen and C. Jin, "Reading it like an open book: Single-trace blind side-channel attacks on garbled circuit frameworks," in *Annual Computer Security Applications Conference, ACSAC 2024, Honolulu, HI, USA, December 9-13, 2024*. IEEE, 2024, pp. 410–424.

[18] B. David, J. Drake, D. Khovratovich, M. Maller, H. Montgomery, C. Orlandi, P. Scholl, O. Shlomovits, and R. Wahby, "The red wedding: Playing attacker in MPC ceremonies," 2021, real World Crypto 2021, contributed talk. [Online]. Available: https://iacr.org/submit/files/slides/2021/rwc/rwc2021/9/slides.pdf

[19] J. van de Pol, "Dont overextend your Oblivious Transfer," 2023, blog post, accessed 14-11-2024. [Online]. Available: https://blog.trailofbits.com/2023/09/20/dont-overextend-your-oblivious-transfer/

[20] N. Makriyannis, O. Yomtov, and A. Galansky, "Practical key-extraction attacks in leading MPC wallets," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. ACM, 2024, pp. 3053–3064.

[21] Y. Lindell, "Fast secure two-party ECDSA signing," in *Advances in Cryptology – CRYPTO 2017, Part II*, ser. Lecture Notes in Computer Science, J. Katz and H. Shacham, Eds., vol. 10402. Springer, Cham, Aug. 2017, pp. 613–644.

[22] D. Bernhard, O. Pereira, and B. Warinschi, "How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios," in *Advances in Cryptology – ASIACRYPT 2012*, ser. Lecture Notes in Computer Science, X. Wang and K. Sako, Eds., vol. 7658. Springer, Berlin, Heidelberg, Dec. 2012, pp. 626–643.

[23] R. Bendlin, I. Damgård, C. Orlandi, and S. Zakarias, "Semi-homomorphic encryption and multiparty computation," in *Advances in Cryptology – EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, K. G. Paterson, Ed., vol. 6632. Springer, Berlin, Heidelberg, May 2011, pp. 169–188.

[24] J. B. Nielsen, P. S. Nordholt, C. Orlandi, and S. S. Burra, "A new approach to practical active-secure two-party computation," in *Advances in Cryptology – CRYPTO 2012*, ser. Lecture Notes in Computer Science, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, Berlin, Heidelberg, Aug. 2012, pp. 681–700.

[25] X. Wang, S. Ranellucci, and J. Katz, "Authenticated garbling and efficient maliciously secure two-party computation," in *ACM CCS 2017: 24th Conference on Computer and Communications Security*, B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds. ACM Press, Oct. / Nov. 2017, pp. 21–37.

[26] ——, "Global-scale secure multiparty computation," in *ACM CCS 2017: 24th Conference on Computer and Communications Security*, B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds. ACM Press, Oct. / Nov. 2017, pp. 39–56.

[27] S. S. Burra, E. Larraia, J. B. Nielsen, P. S. Nordholt, C. Orlandi, E. Orsini, P. Scholl, and N. P. Smart, "High-performance multi-party computation for binary circuits based on oblivious transfer," *Journal of Cryptology*, vol. 34, no. 3, p. 34, Jul. 2021.

[28] I. Damgård, M. Keller, E. Larraia, V. Pastro, P. Scholl, and N. P. Smart, "Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits," in *ESORICS 2013: 18th European Symposium on Research in Computer Security*, ser. Lecture Notes in Computer Science, J. Crampton, S. Jajodia, and K. Mayes, Eds., vol. 8134. Springer, Berlin, Heidelberg, Sep. 2013, pp. 1–18.

[29] M. Keller, E. Orsini, and P. Scholl, "MASCOT: Faster malicious arithmetic secure computation with oblivious transfer," in *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds. ACM Press, Oct. 2016, pp. 830–842.

[30] M. Keller, V. Pastro, and D. Rotaru, "Overdrive: Making SPDZ great again," in *Advances in Cryptology – EUROCRYPT 2018, Part III*, ser. Lecture Notes in Computer Science, J. B. Nielsen and V. Rijmen, Eds., vol. 10822. Springer, Cham, Apr. / May 2018, pp. 158–189.

[31] C. Baum, D. Cozzo, and N. P. Smart, "Using TopGear in overdrive: A more efficient ZKPoK for SPDZ," in *SAC 2019: 26th Annual International Workshop on Selected Areas in Cryptography*, ser. Lecture Notes in Computer Science, K. G. Paterson and D. Stebila, Eds., vol. 11959. Springer, Cham, Aug. 2019, pp. 274–302.

[32] R. Cramer, I. Damgård, D. Escudero, P. Scholl, and C. Xing, "SPD $\mathbb{Z}_{2^k}$: Efficient MPC mod $2^k$ for dishonest majority," in *Advances in Cryptology – CRYPTO 2018, Part II*, ser. Lecture Notes in Computer Science, H. Shacham and A. Boldyreva, Eds., vol. 10992. Springer, Cham, Aug. 2018, pp. 769–798.

[33] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *42nd Annual Symposium on Foundations of Computer Science*. IEEE Computer Society Press, Oct. 2001, pp. 136–145.

[34] R. Cleve, "Limits on the security of coin flips when half the processors are faulty (extended abstract)," in *18th Annual ACM Symposium on Theory of Computing*. ACM Press, May 1986, pp. 364–369.

[35] M. Hastings, B. Hemenway, D. Noble, and S. Zdancewic, "SoK: General purpose compilers for secure multi-party computation," in *2019 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2019, pp. 1220–1237.

[36] X. Fan, K. Chen, G. Wang, M. Zhuang, Y. Li, and W. Xu, "NFGen: Automatic non-linear function evaluation code generator for general-purpose MPC platforms," in *ACM CCS 2022: 29th Conference on Computer and Communications Security*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM Press, Nov. 2022, pp. 995–1008.

[37] D. Lu and A. Kate, "Rpm: Robust anonymity at scale," *Proceedings on Privacy Enhancing Technologies*, 2023.

[38] B. C. Ghosh, S. Patranabis, D. Vinayagamurthy, V. Ramakrishna, K. Narayanam, and S. Chakraborty, "Private certifier intersection," in *ISOC Network and Distributed System Security Symposium – NDSS 2023*. The Internet Society, Feb. 2023.

[39] I. Damgård, J. B. Nielsen, M. Nielsen, and S. Ranellucci, "The TinyTable protocol for 2-party secure computation, or: Gate-scrambling revisited," in *Advances in Cryptology – CRYPTO 2017, Part I*, ser. Lecture Notes in Computer Science, J. Katz and H. Shacham, Eds., vol. 10401. Springer, Cham, Aug. 2017, pp. 167–187.

[40] P. Rogaway and T. Shrimpton, "A provable-security treatment of the key-wrap problem," in *Advances in Cryptology – EUROCRYPT 2006*, ser. Lecture Notes in Computer Science, S. Vaudenay, Ed., vol. 4004. Springer, Berlin, Heidelberg, May / Jun. 2006, pp. 373–390.

[41] I. Damgård, K. Damgård, K. Nielsen, P. S. Nordholt, and T. Toft, "Confidential benchmarking based on multiparty computation," in *FC 2016: 20th International Conference on Financial Cryptography and Data Security*, ser. Lecture Notes in Computer Science, J. Grossklags and B. Preneel, Eds., vol. 9603. Springer, Berlin, Heidelberg, Feb. 2016, pp. 169–187.

[42] D. Hofheinz and J. Müller-Quade, "Universally composable commitments using random oracles," in *TCC 2004: 1st Theory of Cryptography Conference*, ser. Lecture Notes in Computer Science, M. Naor, Ed., vol. 2951. Springer, Berlin, Heidelberg, Feb. 2004, pp. 58–76.

[43] M. Blanton, D. Murphy, and C. Yuan, "Efficiently compiling secure computation protocols from passive to active security: Beyond arithmetic circuits," *Proceedings on Privacy Enhancing Technologies*, vol. 2024, no. 1, pp. 74–97, Jan. 2024.

[44] Y. Sang, N. Luo, S. Judson, B. Chaimberg, T. Antonopoulos, X. Wang, R. Piskac, and Z. Shao, "Ou: Automating the parallelization of zero-knowledge protocols," in *ACM CCS 2023: 30th Conference on Computer and Communications Security*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM Press, Nov. 2023, pp. 534–548.

[45] R. Canetti and T. Rabin, "Universal composition with joint state," in *Advances in Cryptology – CRYPTO 2003*, ser. Lecture Notes in Computer Science, D. Boneh, Ed., vol. 2729. Springer, Berlin, Heidelberg, Aug. 2003, pp. 265–281.

# Appendix A. Batch MAC Check Protocol

In Figure 8, we present the batch MAC checking protocol used in SPDZ [28]. Normally, the protocol is described using an ideal functionality for coin-tossing. Here, we instead explicitly write how the coin-tossing is implemented using a commitment functionality, as this is relevant for the attack in Section 6.2.1.

$\Pi_{\mathsf{BatchCheckMAC}}(x_1, \ldots, x_m, [\![x_1]\!], \ldots, [\![x_m]\!])$

The protocol uses a pseudorandom generator $G : \{0,1\}^\lambda \to \mathbb{F}^m$, where $\lambda$ is the security parameter.

1) Each party $P_i$ samples a random seed $s_i \leftarrow \{0,1\}^\lambda$, and broadcasts a commitment to $s_i$
2) After receiving all other parties' commitments, each party opens their commitment to reveal $s_i$
3) If any opening fails, abort.
4) Compute $s = \bigoplus s_i$
5) Compute $(\chi_1, \ldots, \chi_m) = G(s)$
6) Compute $[\![\tilde{x}]\!] = \sum_{i=1}^m \chi_i \cdot [\![x_i]\!]$
7) Run $\mathsf{CheckMAC}(\tilde{x}, [\![\tilde{x}]\!])$ (see Figure 1)

Figure 8: SPDZ batch MAC checking protocol using coin-tossing

We note that, if the seed $s$ can be predicted by the adversary ahead of time, it is easy to cheat the protocol and pass the check with incorrect opened values $x'_1, \ldots, x'_m$. Concretely, an adversary can cheat however he likes in the openings of $x_1, \ldots, x_{m-1}$ by opening them to some $x'_1, \ldots, x'_{m-1}$. Then, using knowledge of the seed $s$ and the $\chi_i$'s, he can pick $x'_m$ such that

$$\sum_{i=1}^{m} \chi_i \cdot x'_i = \sum_{i=1}^{m} \chi_i \cdot x_i$$

and open $x_m$ to $x'_m$, to ensure that the check passes. This is possible as long as $\chi_m \neq 0$, which holds with probability $1 - 1/|\mathbb{F}|$ if $G$ is a secure PRG.

# Appendix B. Modeling Multi-Threaded SPDZ with Shared Preprocessing

In this section we aim to design a new ideal MPC functionality and protocol, one which incorporates multi-threaded capabilities similar to MP-SPDZ. We sketch a model where each thread of computation is represented by a unique and independent ideal online functionality. We then show that a modified SPDZ protocol with masked MAC checks (see Section 5.2.2) can securely implement the ideal functionality, allowing multiple threads to share the same MAC key and preprocessing material.

**Functionality $\mathcal{F}_{\mathsf{Prep}}$**

Usage: The following macro is used to construct $[\![\cdot]\!]$ representations.

$\mathsf{Bracket}(x^{(1)}, \ldots, x^{(n)}, \Delta)$

1) Compute the MAC $\tau_x = \alpha \cdot x + \Delta$, where $x = \sum_{i=1}^{n} x^{(i)}$
2) For each corrupt party $P_j$, let the environment specify share $\tau_x^{(j)}$
3) For each honest party $P_j$, pick $\tau_x^{(j)}$ uniformly at random, such that $\tau_x = \sum_{i=1}^{n} \tau_x^{(i)}$
4) Send $(x^j, \tau_x^{(j)})$ to each honest party $P_j$.

**Initialise:** On input $(\mathsf{initialise}, \mathbb{F})$ from all parties, store the finite field $\mathbb{F}$. On message fail from the environment, send fail to all honest parties and stop, otherwise continue:

1) For each corrupt party $P_j$, let the environment pick the share $\alpha^{(j)}$
2) For each honest party $P_j$, pick $\alpha^{(j)}$ uniformly and send to $P_j$
3) Set $\alpha = \sum_{i=1}^{n} \alpha^{(i)}$

**Single:** On input $(\mathsf{single}, i)$ from all parties. On message fail from the environment, send fail to all honest parties and stop, otherwise do the following:

1) For each corrupt party $P_j$, let the environment pick $r^{(j)}$
2) For each honest party $P_j$, pick $r^{(j)}$ uniformly at random.
3) Let environment pick $\Delta_r$
4) Run the macro $\mathsf{Bracket}(r^{(1)}, \ldots, r^{(n)}, \Delta_r)$ and send $r = \sum_i r^{(i)}$ to party $P_i$.

**Triple:** On input $(\mathsf{triple})$ from all parties. On message fail from the environment, send fail to all honest parties and stop, otherwise do the following:

1) For each corrupt party $P_j$, let the environment pick $u^{(j)}$, $v^{(j)}$ and $w^{(j)}$
2) Let the environment pick $\Delta_u, \Delta_v, \Delta_w$
3) For each honest party $P_j$, pick $u^{(j)}$ and $v^{(j)}$ uniformly at random.
4) Define $u = \sum_{i=1}^{n} u^{(i)}$, $v = \sum_{i=1}^{n} v^{(i)}$ and $w = u \cdot v$
5) For each honest party $P_j$, pick $w^{(j)}$ uniformly at random such that $w = \sum_{i=1}^{n} w^{(j)}$
6) Run the macros $\mathsf{Bracket}(u^{(1)}, \ldots, u^{(n)}, \Delta_u)$, $\mathsf{Bracket}(v^{(1)}, \ldots, v^{(n)}, \Delta_v)$ and $\mathsf{Bracket}(w^{(1)}, \ldots, w^{(n)}, \Delta_w)$

Figure 9: Ideal functionality for the preprocessing phase of SPDZ. All arithmetic is in $\mathbb{F}_p$.

## B.1. Modeling Parallel Threads

**First Attempt: Multi-session online phase UC.** To capture parallel execution of MPC, the most natural approach is to consider a protocol $\pi$ in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid model, where parties invoke multiple instances of the ideal MPC functionality, one for each thread. The composition theorem guarantees that the security of each MPC functionality is maintained even when multiple instances are run concurrently. When implementing each separate instance of $\mathcal{F}_{\mathsf{ABB}}$, one can use the SPDZ online protocol with $\Pi_{\mathsf{Online}}$ in the $\mathcal{F}_{\mathsf{Prep}}$-hybrid model, where $\mathcal{F}_{\mathsf{Prep}}$ is the SPDZ preprocessing functionality, shown in Fig. 9. Since each session uses an independent session of $\mathcal{F}_{\mathsf{Prep}}$, they have independently sampled MAC keys and must each generate their own set of multiplation triples independently. Clearly, while this approach is secure, it does not accurately capture the setting of MP-SPDZ or SCALE-MAMBA, where we can perform independent computation in different threads using the same MAC key.

**Formalising a multi-session online phase in JUC.** To overcome the limitations on state sharing between protocols, we can use the framework for universal composition with joint state, also called the JUC framework [45]. JUC is a UC variant that allows a limited form of state sharing. Then, we can use the JUC theorem to reason about the security of the protocol when the MPC sessions share a "joint state", namely the MAC key.

To use JUC, the first step is to define a multi-session extension $\hat{\mathcal{F}}_{\mathsf{ABB}}$ of $\mathcal{F}_{\mathsf{ABB}}$, which essentially behaves the same way as multiple independent copies of $\mathcal{F}_{\mathsf{ABB}}$. $\hat{\mathcal{F}}_{\mathsf{ABB}}$ can be defined based on $\mathcal{F}_{\mathsf{ABB}}$, according to [45], roughly as follows:

- In addition to the session identifier SID, all valid messages include a sub-session identifier SSID.
- Each sub-session corresponds to a separate copy of $\mathcal{F}_{\mathsf{ABB}}$ (running within $\hat{\mathcal{F}}_{\mathsf{ABB}}$).
- Each message of the form $(sid, ssid, v)$ is passed to the sub-session with SSID $ssid$, if one exists. Otherwise, the message is passed to a new copy of

**Protocol $\hat{\Pi}_{\mathsf{ABB}}$**

Use the notation $[\![x]\!]_T$ for secret shared variable $x$ created in thread $T$.

**Initialize:** Given as input thread $T$. If it is the first time initialize is called, parties invoke $\mathcal{F}_{\mathsf{Prep}}$ with input $(\mathsf{initialise}, \mathbb{F})$ to obtain shares of a MAC key $\langle\alpha\rangle$.
Set thread $T$ as active and invoke the preprocessing functionality to obtain sufficiently many input masks $(r_i, [\![r_i]\!]_T)$ and multiplication triples $([\![u]\!]_T, [\![v]\!]_T, [\![w]\!]_T)$.

**Input:** For party $P_j$ with input $(x_i, T)$ where $T$ is an active thread. Parties select randomness $(r_i, [\![r_i]\!]_T)$ from $\mathcal{F}_{\mathsf{Prep}}$ and do the following:

1) $P_j$ broadcasts the value $\delta = x_i - r_i$
2) Parties compute $[\![x_i]\!]_T = [\![r_i]\!]_T + \delta$

**Add:** Given as input $[\![x]\!]_T, [\![y]\!]_T$ where $T$ is active, parties locally compute

$$[\![x + y]\!]_T = [\![x]\!]_T + [\![y]\!]_T$$

**Multiply:** Given as input $[\![x]\!]_T, [\![y]\!]_T$ where $T$ is active, parties take a fresh multiplication triple $([\![u]\!]_T, [\![v]\!]_T, [\![w]\!]_T)$ from $\mathcal{F}_{\mathsf{Prep}}$ and do the following:

1) Partially open $[\![d]\!]_T = [\![x]\!]_T - [\![u]\!]_T$
2) Partially open $[\![e]\!]_T = [\![y]\!]_T - [\![v]\!]_T$
3) Compute $[\![x \cdot y]\!]_T = [\![w]\!]_T + e \cdot [\![u]\!]_T + d \cdot [\![v]\!]_T + d \cdot e$

**Output:** Given output value $[\![y]\!]_T$ where $T$ is active:

1) Perform MAC check $\Pi_{\mathsf{MaskedCheck}}$ on previous partially opened values on the form $[\![\cdot]\!]_T$. If the check fails, set thread $T$ as inactive, output $\perp$ and skip the next step.
2) Open $[\![y]\!]_T$ and execute $\Pi_{\mathsf{MaskedCheck}}$. If the check fails, set thread $T$ as inactive and output $\perp$.

Figure 10: Protocol for multithreaded online phase of SPDZ, over a finite field $\mathbb{F}$.

$\mathcal{F}_{\mathsf{ABB}}$.

In essence, this gives a functionality almost the same as $\mathcal{F}_{\mathsf{ABB}}$, except that two variables initialised with different SSIDs cannot be added or multiplied. Furthermore, if any sub-session aborts, the remaining sub-sessions are unaffected.

## B.2. Implementing $\hat{\mathcal{F}}_{\mathsf{ABB}}$

We now want to construct a protocol $\hat{\Pi}_{\mathsf{ABB}}$ emulating $\hat{\mathcal{F}}_{\mathsf{ABB}}$, using the original SPDZ online phase as a basis. To prevent the MAC key leakage attack from Section 4, the new online phase will employ the alternative MAC check subprotocol $\Pi_{\mathsf{MaskedCheck}}$, described in Figure 4, which does not leak information about the MAC key share. The specification of $\hat{\Pi}_{\mathsf{ABB}}$ is given by Figure 10. Note that $\hat{\Pi}_{\mathsf{ABB}}$ requires each thread to use its own portion of preprocessing data (multiplication triples etc.) from $\mathcal{F}_{\mathsf{Prep}}$.

## B.3. Security Analysis

We can now argue that a SPDZ with the masked MAC check in a simple multi-threaded setting is JUC-secure.

**Theorem 1.** *$\hat{\Pi}_{\mathsf{ABB}}$ implements $\hat{\mathcal{F}}_{\mathsf{ABB}}$ in the $\mathcal{F}_{\mathsf{Prep}}$-hybrid model with statistical security against any static, active adversary corrupting up to $n - 1$ parties.*

*Proof (sketch).* Most of the arguments are identical to the proof of $\mathcal{F}_{\mathsf{ABB}}$. We construct a simulator $\hat{\mathcal{S}}_{\mathsf{ABB}}$ such that for any environment, interacting with $\hat{\Pi}_{\mathsf{ABB}}$ is indistinguishable from interacting with $\hat{\mathcal{F}}_{\mathsf{ABB}}$ and the simulator. See Figure 11 for the specification of $\hat{\mathcal{S}}_{\mathsf{ABB}}$.

Recall that the environment sees the entire view of the corrupt parties and the output of the honest parties. In the initialize step, the simulator learns the shares of the MAC key, multiplication triples and input randomness chosen by the corrupt parties. The $\delta$ revealed in the input phase is uniformly random and thus has the same distribution in both the real and simulated interaction. Similarly, in the multiplication step, the values $e, d$ are uniformly random when $u, v$ is unknown and, as such, easily simulated. For the MAC check in step 1 of the output phase, using the revised MAC check protocol, all the values exchanged are completely random, as shown in Lemma 1. In step 2 of the output phase, the simulator sends shares and MACs consistent with the output $y$. As a result, when the output in the real and simulated process is the same, the views are indistinguishable.

Next, we need to argue that corrupt parties are only able to create a forgery, i.e., affect the output of the protocol without being caught, with negligible probability. Changing the output without being caught corresponds to guessing the uniformly random MAC key $\alpha$ as argued in Lemma 1. This

**Simulator $\hat{\mathcal{S}}_{\mathsf{ABB}}$**

**Initialize:** The first time initialize is called, perform the steps of *initialize* in $\mathcal{F}_{\mathsf{Prep}}$. Run the single and triple command of $\mathcal{F}_{\mathsf{Prep}}$ the desired number of times to obtain multiplication triples and input randomness. Note that the simulator receives shares and error terms from corrupt parties.

**Input:** On input from honest party $P_i$, simulate input step honestly with dummy input. If $P_i$ is corrupted, perform the input steps honestly. When $\delta$ is broadcasted, compute $x_i = r_i + \delta$ and send to $\hat{\mathcal{F}}_{\mathsf{ABB}}$

**Add:** Emulate the protocol honestly and call *add* on $\hat{\mathcal{F}}_{\mathsf{ABB}}$.

**Multiply:** Emulate the protocol honestly and call *multiply* on $\hat{\mathcal{F}}_{\mathsf{ABB}}$.

**Output:** Perform the MAC checks of step 1 according to the protocol. If any check fails, abort further computation for this subsession. Otherwise, send *output* to $\hat{\mathcal{F}}_{\mathsf{ABB}}$ to retrieve the output $y$. The simulator has shares of a possibly incorrect output $y'$, which was obtained from computation using dummy inputs for honest parties. Adjust the view of an honest party $P_i$ by adding $y - y'$ to the share of the output and $\alpha(y - y')$ to the share of the MAC. Exchange shares of the output with the corrupt parties according to the protocol. If the correct shares were sent by the corrupt parties, send $\mathsf{OK}$ to $\hat{\mathcal{F}}_{\mathsf{ABB}}$, otherwise send $\mathsf{abort}$ to $\hat{\mathcal{F}}_{\mathsf{ABB}}$.

Figure 11: Simulator for $\hat{\mathcal{F}}_{\mathsf{ABB}}$.

happens with probability $1/|\mathbb{F}|$. Corrupt parties get to try and guess the MAC key in every thread, but the number of threads is still bounded by a polynomial, and so the error probability remains negligible. □

## B.4. Applying the JUC Theorem

Once we have shown how to implement the multi-session extension $\hat{\mathcal{F}}_{\mathsf{ABB}}$ with a protocol $\hat{\Pi}_{\mathsf{ABB}}$, the JUC theorem allows us to obtain a protocol that realises any number of independently copies of $\mathcal{F}_{\mathsf{ABB}}$, using only the protocol $\hat{\Pi}_{\mathsf{ABB}}$, where each underlying implementation may share internal state. Concretely, given any protocol $\pi$ in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid model, JUC defines a composed protocol, $\pi^{[\hat{\Pi}_{\mathsf{FABB}}]}$, which only makes calls to $\hat{\Pi}_{\mathsf{ABB}}$ running in the $\mathcal{F}_{\mathsf{Prep}}$-hybrid model.

**Corollary 1.** *Any protocol $\pi$ in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid model is implemented by $\pi^{[\hat{\Pi}_{\mathsf{ABB}}]}$ in the $\mathcal{F}_{\mathsf{Prep}}$-hybrid model.*

The corollary follows directly from Theorem 1 and the JUC theorem. We stress that the security of the ideal functionality $\mathcal{F}_{\mathsf{ABB}}$ is guaranteed by the regular UC theorem, even when multiple instances are used concurrently by some arbitrary protocol $\pi$. The above corollary then gives security guarantees for the composed protocol $\pi^{[\hat{\Pi}_{\mathsf{ABB}}]}$.

## B.5. Discussion and Limitations of the Model

Our JUC modeling of SPDZ with the masked MAC check allows multiple, independent instances of the SPDZ online phase to be run, while using the same MAC key and preprocessing material (provided this material is appropriately divided among the instances such that it is never reused). This models a setting where concurrent implementations of the online phase are being run, similar to multiple threads running concurrently in the MP-SPDZ online phase.

However, our model does not capture some of the features of MP-SPDZ, since our sessions cannot share any state beyond what is output by the preprocessing phase. In particular, secret-shared values cannot be passed from one thread to another. However, we can allow *public* values to be opened in one thread, and then transferred to another thread (via the high-level protocol $\pi$) to influence computations there.